LF-Filter: Differentiated Processing for Intra-Flow Packet Delay Monitoring in High-Speed Data Streams

Xiang Chen*
The State Key Laboratory of Blockchain and Data Security
College of Computer Science and Technology
Zhejiang University
Hangzhou, Zhejiang, China
wasdnsxchen@gmail.com

Zhengli Lin*
Fuzhou University
Fuzhou, Fujian, China
linzhengli0911@gmail.com

Lida Liao
Fuzhou University
Fuzhou, Fujian, China
241020021@fzu.edu.cn

Longlong Zhu
Zhejiang University
Hangzhou, Zhejiang, China
kikionepin@gmail.com

Linying Zheng
Zhejiang University
Hangzhou, Zhejiang, China
zhenglinying0106@gmail.com

Jiashuo Yu
Zhejiang University
Hangzhou, Zhejiang, China
joshuayu.2048@gmail.com

Yimeng Liu
Zhejiang University
Hangzhou, Zhejiang, China
liuyimeng@zju.edu.cn

Jiajie Su
Zhejiang University
Hangzhou, Zhejiang, China
sujiajie@zju.edu.cn

Yiheng Xu
Zhejiang University
Hangzhou, Zhejiang, China
12221238@zju.edu.cn

Pengchao Yao
Zhejiang University
Hangzhou, Zhejiang, China
pcyao@zju.edu.cn

Hongyan Liu
Fuzhou University
Fuzhou, Fujian, China
hyliu20@zju.edu.cn

Dong Zhang
Fuzhou University
Fuzhou, Fujian, China
zhangdong@fzu.edu.cn

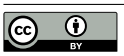
Chunming Wu†
Zhejiang University
Hangzhou, Zhejiang, China
wuchunming@zju.edu.cn

Abstract

Monitoring inter-event delays in high-speed data streams is fundamental to anomaly detection and temporal pattern discovery. A representative instance in production infrastructures is intra-flow packet delay (IFPD), i.e., the time interval between consecutive packets of the same flow. It serves as a critical metric for detecting network anomalies such as low-rate DDoS attacks and port scans. Existing solutions suffer from high collision rates and fixed temporal granularity, leading to limited detection accuracy.

In this paper, we propose LF-Filter, a novel sketch-based data structure for accurate and efficient IFPD monitoring. The key idea of LF-Filter is to provide differentiated treatment for different flows. In detail, LF-Filter incorporates insights from streaming data structures and proposes four techniques, including (1) flow-aware processing that directs heavy flows to collision-resistant filters, (2) fingerprint-enhanced window filters that reduce false positives by a factor of $2^{f \cdot k}$, (3) a layered window structure with adaptive IFPD monitoring granularity across hot, warm, and cold layers, and (4) prediction-guided query that learns IFPD patterns to accelerate searches. We implement LF-Filter on 32×400 Gbps Tofino2 switches and NVIDIA BlueField3 DPUs. Extensive experiments with production workloads show that compared with state of the arts, LF-Filter improves the F1 score by up to 2.22× under the same memory, and brings benefits to applications varying from arrival jitter detection, APT detection, to caching optimizations. The code is released at https://github.com/lindeaccount2/LF-Filter.

*Xiang Chen and Zhengli Lin contributed equally to this research.
†Chunming Wu is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

KDD '26, Jeju Island, Republic of Korea

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817702

CCS Concepts

• **Networks** → **Network measurement**; **Network monitoring**; *Network performance analysis*; • **Information systems** → *Data stream mining*.

Keywords

Inter-Event Delay Monitoring; Sketch; Streaming Processing

ACM Reference Format:

Xiang Chen, Zhengli Lin, Lida Liao, Longlong Zhu, Linying Zheng, Jiashuo Yu, Yimeng Liu, Jiajie Su, Yiheng Xu, Pengchao Yao, Hongyan Liu, Dong Zhang, and Chunming Wu. 2026. LF-Filter: Differentiated Processing for Intra-Flow Packet Delay Monitoring in High-Speed Data Streams. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3770855.3817702

1 Introduction

Monitoring inter-event delays in high-speed data streams is fundamental to anomaly detection and temporal pattern discovery. In typical streaming scenarios, such as network traffic analysis [23] and user behavior tracking [17, 24], identifying abnormal time gaps between consecutive events of the same stream enables early detection of security threats, performance degradation, and behavioral anomalies [1]. However, the large volume and speed of modern data streams exclude the strawman approach of storing the entire event histories, leading to the trend of developing compact probabilistic data structures with bounded memory [13, 14, 22, 34, 35, 40].

Among high-speed data streams in production, network traffic is a particularly challenging case. A representative inter-event delay metric is *intra-flow packet delay* (IFPD) [41], which denotes the time interval between consecutive packets of the same flow in network traffic. Unlike traditional per-hop or end-to-end delays that provide only aggregate statistics, IFPD captures detailed traffic arrival patterns at the individual packet level. This fine-grained visibility enables a wide range of applications: (1) *Arrival jitter detection* [27] identifies flows with frequently changing IFPDs, which is crucial for maintaining QoS in real-time applications. (2) *Advanced persistent threat (APT) detection* [7] discovers attack patterns characterized by long intra-flow packet intervals. (3) *Caching optimization* [10] uses IFPD patterns to improve cache replacement policies.

However, prior solutions for IFPD detection suffer from either poor accuracy or high memory overhead. Specifically, traditional delay monitoring methods provide only coarse-grained aggregate statistics. They fail to capture individual packet arrivals inside flows. For per-flow timestamp tracking, sketches such as Delay Sketch [41] suffer from hash collisions that evict fingerprints and timestamps. Also, their fingerprint overhead further limits the number of maintainable entries. Bloom filters also exhibit inherent drawbacks. For example, timeout BF [19] uses the minimum window ID across mapped positions, leading to bias on older timestamps and overestimated IFPDs. HyperBF [26] replaces old timestamps to resolve collisions, while a flow's timestamp may be overwritten by unrelated flows and result in information loss.

Recently, FD-Filter [15] attempts to address the above issues with compact memory, but still exhibits fundamental limitations. In detail, it partitions time into consecutive windows and employs window filters to approximately store timestamps, thus improving accuracy. But this design comes with four limitations. First, it treats all flows uniformly regardless of their frequency, which ignores the heavy-tailed nature of network traffic where a few heavy flows contribute most packets [3]. This leads to insufficient accuracy for important heavy flows while wasting resources on light flows. Second, its per-bit OR-based update and AND-based query are prone to hash collisions, i.e., when multiple flows map to the same counter, the window IDs are collided, which increases both false positives and negatives. Third, FD-Filter employs fixed window granularity across all time ranges. It overlooks the fact that recent timestamps near the threshold require finer inspection than older ones. Fourth, its sequential query process searches window filters from the most recent to the oldest. But for high-IFPD packets, their predecessors appeared in distant windows. Thus, such searching leads to unnecessary overhead.

In this context, we aim to answer the following question: *can we achieve accurate and low-overhead IFPD monitoring*? Our answer is **LF-Filter**, i.e., Layered Flow-aware Filter, a novel data structure for accurate and low-overhead IFPD monitoring. Our key idea is that *different flows deserve differentiated processing*. With this idea in mind, our design of LF-Filter revisits the literature of streaming data structures and draws insights (§2.3) to match the specific data streaming case of IFPD monitoring in production networks.

More precisely, LF-Filter integrates four novel techniques into a hierarchical architecture that allocates resources adaptively to the IFPD monitoring of different flows. (1) LF-Filter classifies incoming flows into heavy and light flows. Heavy flows, which contribute most packets to traffic and are more likely to trigger high-IFPD events, are processed in collision-resistant fingerprint-enhanced filters. Instead, light flows use standard filters to save memory. (2) The fingerprint-enhanced filters augment each counter slot with a compact f-bit fingerprint that identifies the flow that occupies that slot. These fingerprints enable LF-Filter to distinguish correct matches from hash collisions, thereby reducing the false positive rate by a factor of $2^{f \cdot k}$, where k is the number of hash functions. (3) LF-Filter organizes window filters into a three-layer structure, including a hot layer for recent timestamps with fine granularity, a warm layer for intermediate timestamps, and a cold layer for older timestamps with coarse granularity. This layered structure provides high accuracy with bounded memory. (4) A lightweight predictor learns each flow's IFPD patterns using exponential moving averages and suggests the most possible layer that contains the record, enabling LF-Filter to skip irrelevant layers and accelerate queries.

In summary, we make the following contributions:

- We identify the limitations of existing IFPD monitoring solutions and propose LF-Filter, a layered flow-aware data structure that addresses these limitations via differentiated processing (§2).
- We design four novel techniques, i.e., flow-aware dual-path processing, fingerprint-enhanced window filters, layered window structure, and prediction-guided query, to collectively improve accuracy, memory consumption, and query speed (§3).
- We provide theoretical analysis of LF-Filter's space complexity, time complexity, false positive and negative rates, and estimation error bounds. We also provide guidelines for optimal parameter configuration in LF-Filter (§4).
- We implement LF-Filter on a testbed of 32×400 Gbps Tofino2 switches and NVIDIA BlueField3 DPUs. Extensive experiments show that LF-Filter achieves 1.69×-2.22× F1 improvements over existing solutions across diverse applications and settings (§5).

2 Motivation

2.1 Problem

Problem formulation. We consider incoming traffic as a packet stream. Each packet is a tuple (x, t), where x is the flowkey (e.g., 5-tuple) and t is the arrival timestamp. Packets sharing the same flowkey x form a flow and are recorded in arrival order. For a flow x with p packets, we represent it as:

$$f_x = ((x, t_1), (x, t_2), \ldots, (x, t_p)) \tag{1}$$

where t_i denotes the arrival time of the i-th packet, $\forall i < j, t_i < t_j$.

Definition 1 (Intra-Flow Packet Delay). *The intra-flow packet delay (IFPD) of the i-th packet in flow x is defined as:*

$$D_i^x = \begin{cases} t_i - t_{i-1} & if\, i > 1 \\ 0 & if\, i = 1 \end{cases} \tag{2}$$

Given an IFPD threshold θ, our goals are two-fold, i.e., (1) detecting high-IFPD packets, whose IFPD exceeds θ, and (2) accurately estimating the IFPD value for high-IFPD packets, where per-packet IFPDs can be further aggregated into flow-level statistics such as mean delay or jitter for applications.

Applications. IFPD captures fine-grained traffic arrival patterns that enable various network applications beyond the scope of traditional metrics. We highlight three representative use cases:

Arrival jitter detection. Arrival jitters quantify the variation in packet arrival time within a flow. They are critical for QoS in real-time applications [6, 27], such as VoIP [12] and video streaming [29], that require stable inter-packet intervals to avoid buffer overflows and underflows. Arrival jitters occur with frequently or significantly changing IFPDs. By monitoring IFPDs, administrators can identify which flows are experiencing jitters and locate bottlenecks.

Advanced persistent threat (APT) detection. APTs [2, 7] are sophisticated attacks that intentionally introduce long idle delays between packets to evade detection. A typical APT has bursts of low-IFPD packets separated by high-IFPD intervals, and repeatedly occurs over time. By monitoring IFPDs, security teams can identify suspicious events that imply APTs. While they need additional analysis to locate APTs from suspicious events, IFPD-based detection acts as a first-line defense and shortens the detection time.

Caching optimization. Caching [10, 16, 39] improves application performance by storing frequently accessed items closer to users. Traditional cache replacement policies such as least frequently used (LFU) may maintain the stale items that are no longer popular. Instead, IFPD offers temporal access patterns. By detecting high IFPDs, caching systems detect low popularity on stale items and proactively evict them to make room for emerging hot ones.

Design goals. According to the applications, we identify three goals when designing an algorithm for monitoring IFPDs:

- **High measurement accuracy.** The algorithm should accurately distinguish high-IFPD packets from low-IFPD ones.
- **Low memory overhead.** The algorithm should avoid storing all flowkeys and timestamps to prevent saturating device memory.
- **High processing speed.** The algorithm should support fast updates and queries to match line-rate traffic.

2.2 Limitations of Existing Solutions

Traditional packet delay monitoring. Traditional methods focus on measuring delays between network devices or hops. They provide aggregate statistics rather than per-packet granularity. More precisely, we categorize existing methods into three categories: (1) *Aggregated packet delay* methods such as LDA [18] inject probe packets and compute average delays across multiple flows.(2) *Per-flow packet delay* methods like RLI [21] estimate flow-level average delays via tracepoints. (3) *Sampling-based* methods, including OPA [33] and INT [28], sample packets due to resource constraints.

However, these approaches are fundamentally unsuitable for IFPD monitoring. Aggregated methods report cross-flow averages that hide per-flow variations. Per-flow methods produce flow-level aggregates that mask individual packet delays. Sampling methods inherently miss non-sampled packets by design.

Sketches. Sketches [5, 36] are compact structures that hash multiple flows to shared counters, trading bounded errors for memory efficiency. Count-min sketch [5] estimates item frequency with the minimum count across mapped positions. Elastic sketch [36] separates heavy and light flows into two parts to adapt to varying traffic. HeavyGuardian [35] employs a similar separation strategy, but uses exponential decay to evict cold items and improve accuracy for heavy hitter detection. HeavyKeeper [11] finds top-k heavy flows while probabilistically removing light flows. WavingSketch [22] provides unbiased frequency estimation by randomly incrementing or decrementing counters. These sketches are primarily designed for frequency estimation and heavy hitter detection. For IFPD monitoring, Delay Sketch [41] stores flow fingerprints alongside timestamps and evicts old entries upon collisions.

However, sketches suffer from two fundamental problems. First, when a collision occurs, existing entries are discarded to create space for new flows, irrespective of importance of the flows that are evicted. As a result, the IFPD monitoring for evicted flows suffers from high false negative rates. Second, storing fingerprints in addition to timestamps brings considerable memory overhead. It reduces the number of entries and exacerbates collisions.

Bloom filters. Bloom filters [4] are bit-array structures for set membership testing. Several extensions track temporal information by partitioning time into windows and recording window IDs. Timeout bloom filter (TOBF) [19] takes the minimum window ID across mapped counters as the timestamp estimate, and HyperBF [26] improves it by replacing old timestamps during collisions.

However, both filters suffer from inherent issues. TOBF's collision accumulation brings bias toward older timestamps. As more flows are inserted, the probability of false timestamps increases, leading to both false positives, i.e., overestimated IFPDs, and false negatives, i.e., overlapped high IFPDs. Also, HyperBF may overwrite a flow's timestamp by irrelevant flows, leading to errors.

FD-Filter. FD-Filter [15] represents the state-of-the-art IFPD monitoring. It introduces both per-bit OR update and AND query that provide better collision handling than direct replacement. It also employs filter recycling to support long-term monitoring and accelerates queries via counter combination.

However, FD-Filter still suffers from four limitations. (1) It applies identical processing to all flows regardless of their importance.

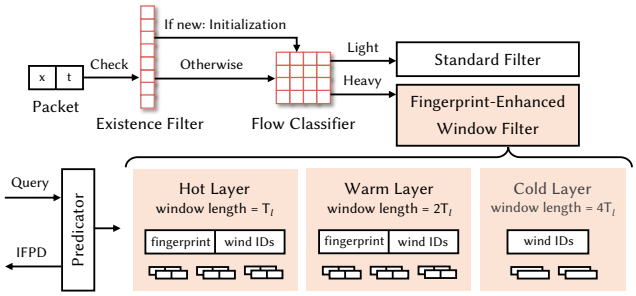


Figure 1: Architecture of LF-Filter. Incoming packets are classified by the existence filter and flow classifier. Heavy flows are processed with fingerprint-enhanced window filters, while light flows use standard filters. The predictor module guides queries to the most likely layer.

It wastes resources on light flows while providing insufficient accuracy for heavy flows that contribute most packets. (2) While per-bit operations outperform direct replacement, OR-ing window IDs from collided flows yields corrupted values, rapidly increasing the false positive rate. (3) Uniform window length across all time ranges is suboptimal since recent timestamps require fine granularity for accurate detection, while old timestamps can tolerate coarser granularity. (4) Querying window filters from the most recent to the oldest brings unnecessary overhead for high-IFPD packets, which appear in windows that are far apart.

2.3 Insights from Streaming Data Structures

The above limitations motivate us to revisit the literature on streaming data structures. At its core, IFPD monitoring is an instance of *inter-event delay tracking* in high-speed streams, which is a fundamental problem across diverse domains. According to our analysis on the literature, we derive our insights as follows.

Insight 1: Flow-aware processing. Count-Min [5] and HeavyKeeper [37] efficiently identify high-frequency items in streams. A key insight is that network traffic follows heavy-tailed distributions [3], i.e., a few flows contribute most packets. This motivates *flow-aware processing* that allocates more resources to heavy flows.

Insight 2: Adaptive granularity. The DGIM algorithm [8] maintains sliding window statistics via exponential histograms. It represents older time ranges are represented with coarser granularity. This achieves $O(\frac{1}{\epsilon}\log^2 N)$ space while bounding approximation error. We follow a similar principle, i.e., designing a *layered window structure* that offers fine granularity for recent timestamps and coarser granularity for older ones.

Insight 3: Fingerprint-based collision handling. Cuckoo filters [9] and other fingerprint-augmented structures use compact signatures to reduce false positives from hash collisions. We extend this idea to our window filters by building *fingerprint-enhanced slots* to distinguish correct matches from collisions and reduce errors.

Insight 4: Prediction in streaming systems. Learned indexes [20] and prediction-guided caching [25] demonstrate that lightweight predictors can accelerate data structure operations by exploiting access patterns. We apply this principle with a *prediction-guided query* that learns IFPD patterns to skip irrelevant layers.

3 LF-Filter Design

The key idea of LF-Filter is to provide *differentiated processing* for different flows through a combination of an existence filter, a flow classifier, fingerprint-enhanced and standard window filters across three layers, and a predictor module. Unlike FD-Filter that treats all flows uniformly and uses fixed window granularity, LF-Filter classifies flows into heavy and light flows and organizes window filters into multiple layers with adaptive granularity. For a packet of a flow, LF-Filter determines the layer that records its previous packet and estimates the IFPD. If it exceeds threshold θ, we consider it as a high-IFPD packet. Table 1 summarizes the major notation.

3.1 Data Structure

Flow classifier (leveraging **Insight 1**) is a count-min sketch [5] that consists of d rows of w_{cm} counters. Each row is associated with an independent hash function. For a flowkey x, we increment all mapped counters and estimate its frequency as the minimum counter value. A flow is heavy if its estimated frequency exceeds threshold α, and light otherwise. The classifier is intentionally lightweight since it aims at coarse classification rather than precise frequency estimation. As discussed in §2, this design is motivated by the heavy-tailed distribution of network traffic, where a small fraction of flows contribute most packets [3].

Fingerprint-enhanced window filter (leveraging **Insight 2-3**) extends window filters, which in their standard form store only a w-bit window ID per slot, by augmenting each of the m_w slots with an f-bit fingerprint identifying the flow that occupies the slot, a w-bit window ID storing timestamp information, and an optional c-bit collision counter. The fingerprint is computed via an independent hash. This fingerprint-based design is inspired by Cuckoo filters [9] and enables LF-Filter to distinguish true matches from hash collisions and significantly reduces false positives.

Specifically, LF-Filter adopts layered window structure that organizes fingerprint-enhanced window filters (FEWFs) into three layers: (1) a *hot layer* with n_h filters that use the window length T_l and cover time range $[t_{cur} - 3n_hT_l, t_{cur}]$; (2) a *warm layer* with n_w filters, which window length is $2T_l$; and (3) a *cold layer* with n_c compressed window filters (CWFs), which window length is $4T_l$. Each FEWF can store up to 3 consecutive window IDs in 2-bit fields. The hot layer provides the finest granularity for recent timestamps to enable accurate detection near threshold θ, while the warm and cold layers progressively employ coarser granularity for older timestamps. CWFs in the cold layer only contain 1-bit existence indicators to improve memory efficiency. This adaptive granularity draws inspiration from DGIM's exponential histograms [8], which represent older time ranges with coarser resolution.

Predictor module (leveraging **Insight 4**) builds on a count-min sketch variant where each counter stores a prediction value in $[0, 2]$ that denotes discrete IFPD levels, i.e., 0 for low IFPD and the hot layer, 1 for medium IFPD and the warm layer, and 2 for high IFPD and the cold layer. Updates use exponential moving average with learning rate β: $\hat{l}_{new} = (1-\beta)\cdot\hat{l}_{old} + \beta\cdot l_{observed}$, where $\hat{l}_{new}$, $\hat{l}_{old}$, and $l_{observed}$ represent the new prediction layer, the old layer, and the current layer, respectively. The module only uses 2-4 bits per counter because it aims to provide coarse guidance for query optimization, not precise IFPD estimation. This prediction-guided

Table 1: Major symbols used in the paper.

Notation	Description
θ	High IFPD threshold
x	Flowkey
t_{cur}	Current timestamp
T_l	Length of each window
T_s	Start time of the latest window
N_i	Number of flows in the i-th window filter
n_h, n_w, n_c	Number of filters in hot, warm, and cold layers
m_w	Number of slots in a window filter
m_e	Number of bits in the existence filter
k_w	Number of hash functions per window filter
k_e	Number of hash functions for existence filter
f	Fingerprint length (in bits)
w	Window ID length (in bits)
β	Learning rate for predictor

approach is inspired by learned indexes [20] and exploits temporal locality in flow behavior to accelerate queries.

Existence filter is a standard bloom filter [4] with m_e bits and k_e hash functions. It tracks which flows have been observed and helps identify the first packets of new flows, i.e., IFPD values equal zero.

Optimization. LF-Filter groups the slots at the same position across filters within a layer into a counter group. The slots sharing the same hash index in different filters are stored contiguously in memory, so they can be accessed together by a single hash computation. With this layout, querying all filters in a layer requires only k_w hashes, reducing memory accesses from $O(k_w \cdot n)$ to $O(k_w)$.

3.2 Basic Operations

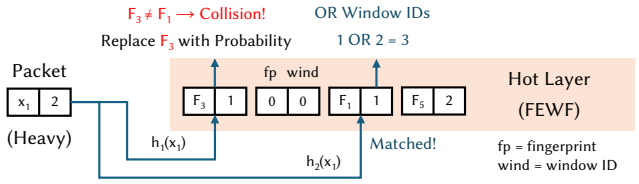
Update. Algorithm 1 summarizes the entire update workflow. To insert a packet (x, t_{cur}), LF-Filter first allocates a new window if the current timestamp exceeds the latest window's range and the current filter already holds 3 window IDs. When no empty filter exists in the hot layer, the oldest filter is moved to the warm layer. This also applies to the movement of warm filters to the cold layer. Next, LF-Filter classifies the flow by querying and incrementing the flow classifier. If the estimated count exceeds α, the flow is heavy; otherwise, it is light. Then LF-Filter updates the existence filter for new flows and updates the appropriate window filter.

For heavy flows, LF-Filter performs *fingerprint-enhanced updates*. Given the flowkey x and window ID $wid = \lfloor t_{cur}/T_l \rfloor \mod 3 + 1$, it computes the fingerprint fp and locates k_w slots via hash functions. For each slot, it considers three cases. (1) If the slot is empty, i.e., $slot.fp = 0$, it directly stores the fingerprint and window ID. (2) If the slot belongs to the same flow, i.e., $slot.fp = fp$, it applies per-bit OR on window IDs, and allows a single slot to track up to 3 window IDs. (3) If the slot is occupied by a different flow, it uses probabilistic replacement, i.e., it replaces the old entry with probability $1/(cnt + 1)$; otherwise, it increments the collision counter, which helps heavy flows retain their slots. For light flows, it performs standard per-bit updates without fingerprints to save memory.

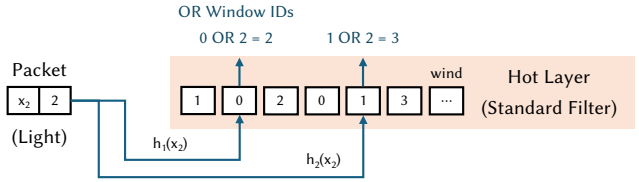
Example (in Figure 2). (a) For a heavy flow packet $(x_1, 2)$ with window ID of 2, LF-Filter inserts into the FEWF in the hot layer. The first hash $h_1(x_1)$ indicates that the fingerprint $F_3 \neq F_1$ causes a collision. So LF-Filter replaces F_3 with F_1 with probability. The second hash $h_2(x_1)$ shows that the fingerprint matches. So LF-Filter performs OR on window IDs. (b) For a light flow packet $(x_2, 2)$, LF-Filter inserts into the standard filter and OR window IDs.

Algorithm 1 LF-Filter Update

Require: Packet (x, t_{cur})
1: **procedure** UPDATE(x, t_{cur})
2: ALLOCATEWINDOW(t_{cur}) ▷ Allocate new window if needed
3: $flow_type \leftarrow$ FlowClassifier.query(x) ▷ Heavy or Light
4: **if** ExistenceFilter.query(x) = false **then** ▷ New flow
5: ExistenceFilter.update(x)
6: **if** $flow_type$ = Heavy **then** ▷ Update filters in hot layer
7: FEWFUPDATE(x, t_{cur}) ▷ Fingerprint-enhanced filters
8: **else**
9: BASICUPDATE(x, t_{cur}) ▷ Basic filters for light flows



(a) Insert a new packet from a heavy flow, Window ID = 2



(b) Insert a new packet from a light flow, Window ID = 2

Figure 2: Example of Update in LF-Filter.

Algorithm 2 LF-Filter Query

Require: Packet (x, t_{cur})
Ensure: Estimated IFPD D, High-IFPD flag
1: **function** QUERY(x, t_{cur})
2: **if** ExistenceFilter.query(x) = false **then** ▷ New flow
3: **return** 0, false
4: $flow_type \leftarrow$ FlowClassifier.query(x)
5: **if** $flow_type$ = Heavy **then**
6: $l \leftarrow$ Predictor.predict(x) ▷ Hot, warm, or cold layer
7: $D \leftarrow$ Search layers starting from l with fingerprints
8: **else**
9: $D \leftarrow$ Search layers from hot, warm, to cold layer
10: Predictor.update(x, observed_layer) ▷ Update prediction
11: **return** D, $(D \geq \theta)$

Query. Algorithm 2 summarizes the entire query workflow. To query the IFPD of packet (x, t_{cur}), LF-Filter first checks the existence filter. If the flow is new, IFPD is zero. Otherwise, it identifies whether the flow is heavy or light, and invokes the corresponding query method as follows.

For heavy flows, it performs *prediction-guided query*. The predictor module returns a prediction level that indicates which layer most likely contains the predecessor packet. If the prediction is below 0.5, LF-Filter searches hot, warm, and cold layers in order; if the prediction is between 0.5 and 1.5, it searches warm, hot, and cold layers in order; otherwise, it searches cold, warm, and hot layers in order. For each layer, it searches filters from newest to oldest. In the fingerprint-enhanced query, it computes fp and locates k_w slots, but only includes the slots where fingerprints match fp in the per-bit AND operation. Upon finding the predecessor, it computes IFPD and updates the predictor with exponential moving average.

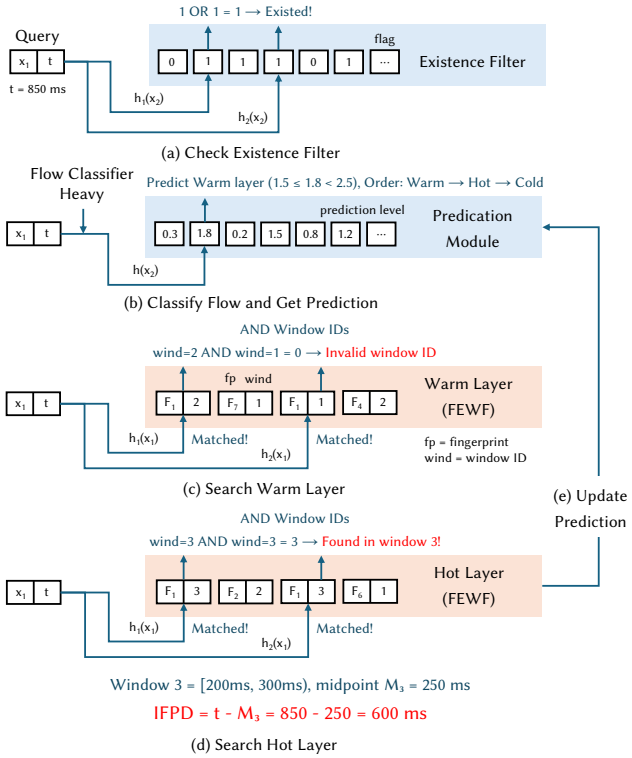


Figure 3: Example of Query in LF-Filter.

For light flows, it performs standard layered query, i.e., searching layers in the fixed order of hot, warm, and cold layers via basic per-bit query without fingerprints. The IFPD is computed by assuming that the predecessor packet arrived at the midpoint of its window, i.e., $\hat{D} = t_{cur} - (T_j + T_l/2)$ for hot layer, $\hat{D} = t_{cur} - (T_j + T_l)$ for warm layer, and $\hat{D} = t_{cur} - (T_j + 2T_l)$ for cold layer, where T_j is the window start time.

Example (in Figure 3). Given a query (x_1, t), where $t = 850$ ms, LF-Filter performs the query as follows. (a) It first checks the existence filter by hashing x_1 to two positions. Since all mapped bits are 1, the flow has been seen before. (b) LF-Filter classifies x_1 as a heavy flow and retrieves its prediction level of 1.8 from the prediction module, which indicates the search order from warm, hot, to the cold layer. (c) According to the order, it searches the warm layer. Both hash positions match the fingerprint, but the AND of window IDs yields 0, indicating an invalid result. (d) Thus, LF-Filter searches the hot layer, where fingerprints match while it finds a valid window ID of 3, which corresponds to [200ms, 300ms) with midpoint $M_3 = 250$ ms. It estimates IFPD as $t - M_3 = 600$ ms. (e) Finally, it updates the prediction module via exponential moving average, e.g., when $\beta = 0.1$, the new prediction $\hat{l}_{new} = (1-\beta) \cdot \hat{l}_{old} + \beta \cdot l_{observed} = (1 - 0.1) \cdot 1.8 + 0.1 \cdot 0 = 1.62$.

Rationale. We explain the advantages of fingerprint-enhanced operations in LF-Filter. When a flow x maps to a slot occupied by another flow y, FD-Filter returns y's window ID, which results in IFPD overestimation or underestimation. In contrast, LF-Filter's fingerprint mismatch identifies the collision, thus excluding the polluted slot from the result. Hence, its false positive rate reduction is significant. FD-Filter's per-slot collision probability is around N/m_w, while in LF-Filter, it becomes $N/(m_w \cdot 2^f)$ with fingerprints, which offers a factor of 2^f improvement.

4 Theoretical Analysis

As shown in Table 2, we theoretically analyze LF-Filter, including complexity, accuracy bounds, and parameter guidelines. Appendix 1 shows detailed proofs.

Complexity. We analyze space and time complexity as follows.

Theorem 1. *The update and query time complexity is $O(k_e + k_w)$.*

Theorem 2. *The space complexity of LF-Filter is $O(m_e + (f + w) \cdot m_w \cdot (n_h + n_w) + m_w \cdot n_c)$.*

False positive rate. A false positive occurs when a low-IFPD packet is wrongly identified as high-IFPD.

Lemma 1 (Packet Arrival Distribution). *For Poisson arrivals with rate λ, if the c-th packet arrives at t_c, the probability that IFPD exceeds v is $p_v = (1 - v/t_c)^c$.*

Lemma 2 (Collision Probability). *For fingerprint-enhanced filters, the effective collision probability, i.e., colliding at the same slot and matching fingerprints, is $P_{col} = \frac{1}{m_w \cdot 2^f}$.*

Theorem 3 (False Positive Rate). *Let N_i be the number of flows in a filter. The false positive rate is:*

$$FPR_{LF} = \left(1 - \left(1 - \frac{1}{m_w \cdot 2^f}\right)^{N_i}\right)^{k_w} \approx \left(\frac{N_i}{m_w \cdot 2^f}\right)^{k_w} \tag{3}$$

Compared to FD-Filter's $FPR_{FD} \approx (N_i/m_w)^{k_w}$, the reduction is:

$$\frac{FPR_{FD}}{FPR_{LF}} = 2^{f \cdot k_w} \tag{4}$$

For $f = 4$ bits and $k_w = 3$, this yields a $2^{12} = 4096\times$ reduction.

False negative rate. A false negative occurs when a high-IFPD packet is wrongly identified as low-IFPD.

Lemma 3 (Layer Hit Probability). *For high-IFPD packets with exponential delay distribution at rate μ, the probabilities of predecessor in each layer are*

$$p_h = 1 - e^{-3\mu n_h T_l}, \quad p_w = e^{-3\mu n_h T_l}(1 - e^{-6\mu n_w T_l}), \quad p_c = 1 - p_h - p_w \tag{5}$$

Theorem 4 (Slot Retention Probability). *Let $\bar{c} = N_i/m_w$ be the average collisions per slot. The probability that a slot retains flow x's information is:*

$$P_{retain} = \left(1 - \frac{1}{m_w}\right)^{N_i - 1} + \frac{1}{m_w} \cdot \left(\frac{1}{2^f} + \left(1 - \frac{1}{2^f}\right) \cdot \frac{1}{\bar{c} + 1}\right) \tag{6}$$

Theorem 5 (False Negative Rate). *The false negative rate of LF-Filter is bounded by:*

$$FNR_{LF} = \sum_{l \in \{h,w,c\}} p_l \cdot (1 - P_{retain,l}^{k_w}) \tag{7}$$

where p_l is from Lemma 3 and $P_{retain,l}$ is from Theorem 4.

Error. We analyze the estimation error of LF-Filter.

Theorem 6 (Expected Estimation Error). *The expected IFPD estimation error is:*

$$\mathbb{E}[\epsilon] = p_h \cdot \frac{T_l}{4} + p_w \cdot \frac{T_l}{2} + p_c \cdot T_l \tag{8}$$

Table 2: Theoretical results and comparison.

Metric	FD-Filter	LF-Filter	Improvement
False Positive Rate	$(N/m)^k$	$(N/(m \cdot 2^f))^k$	$\downarrow 2^{fk}\times$
False Negative Rate	$1 - P_{ret}^k$	$\sum_l p_l(1 - P_{ret,l}^k)$	$\downarrow$
Estimation Error	$T_l/4$	$p_h T_l/4 + p_w T_l/2 + p_c T_l$	$\downarrow$ 30-50%
Query Complexity	$O(k \cdot n)$	$O(k \cdot \mathbb{E}[L])$	$\downarrow$ 1.2-1.5×

where $T_l/4$ is the expected error within a window of length T_l (from midpoint estimation). For $p_h = 0.7$, $p_w = 0.2$, $p_c = 0.1$, and $n_h = 6$, the ARE bound is $\approx 2.5\%$.

Prediction-guided query. The prediction module uses exponential moving average (EMA) to learn flow patterns. Let $\hat{l}_n$ be the predicted layer after n packets and l^* be the true layer. The EMA update $\hat{l}_n = (1-\beta)\hat{l}_{n-1} + \beta l^*$ converges to l^* within $O(\frac{1}{\beta}\ln\frac{2}{\epsilon})$ packets for ϵ-accuracy. With prediction accuracy α, the expected number of searched layers is $\mathbb{E}[L] = \alpha \cdot 1 + (1-\alpha)\cdot 2$, e.g., when $\alpha = 0.85$, $\mathbb{E}[L] = 1.15$ compared to sequential search's $\mathbb{E}[L_{seq}] = p_h + 2p_w + 3p_c \approx 1.4$, which is a 1.22× speedup.

Parameter guidelines. Given the target FPR ϵ and flow count N_i, the filter size should satisfy $m_w \geq N_i \cdot 2^f/\epsilon^{1/k_w}$. Following bloom filter theories, $k_w = 0.693 \cdot m_w \cdot 2^f/N_i$ minimizes FPR while $k_w = 3$ balances accuracy and speed.

5 Evaluation

- (Exp#1-3) LF-Filter brings significant benefits to real-world applications, e.g., improving F1 scores by 20.7% and 12.8% for jitter and APT detection, and improving cache hit rates to 16.8%-80.8%.
- (Exp#4) LF-Filter improves the accuracy of baselines by 1.69×-2.22× when varying memory sizes.
- (Exp#5) LF-Filter maintains high accuracy across varying IFPD thresholds, e.g., achieving an F1 score of 79.9%.
- (Exp#6) LF-Filter shows low sensitivity to parameter changes.
- (Exp#7) Our ablation study shows that our design of individual components collectively enables accurate IFPD monitoring.
- (Exp#8) LF-Filter is resource-efficient. It only consumes 8 stages and a small portion of resources on switch ASICs.
- (Exp#9) LF-Filter achieves Tbps-level throughput on switch ASICs and provides up to 2.79× higher throughput on DPUs.

5.1 Setup

Platform. We evaluate LF-Filter on two devices. First, we implement LF-Filter on the switch ASIC pipelines of a 12.8 Tbps Tofino switch via $P4_{16}$. Second, we run LF-Filter on the SoC cores in a 400 Gbps NVIDIA BlueField3 DPU, which is attached to a server with 36-core Intel Xeon Gold 6240C 2.60 GHz CPU and 128-GB RAM. The operating system uses Linux kernel 5.4.0.

Traces. Since traffic characteristics evolve over time, we collect four backbone traces [31] from 2023 to 2026. While these traces are collected in consecutive years, they record diverse characteristics, e.g., the 2025 trace consists of approximately 1.8 million packets, characterized by multiple traffic bursts. We summarize their statistics in Table 3 and Figure 4. Across all four traces, the top 10% of flows consistently account for 58%-79% of total packets and 83%-100% of high-IFPD events, confirming the heavy-tailed distribution that motivates LF-Filter's differentiated processing.

Table 3: Trace statistics.

Date	Packets	Flows (SrcIP)
2023	1,580,558	351,574
2024	1,671,286	395,211
2025	1,823,279	742,231
2026	1,812,667	845,190

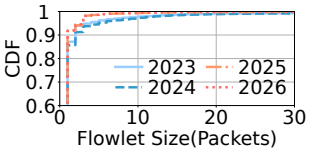
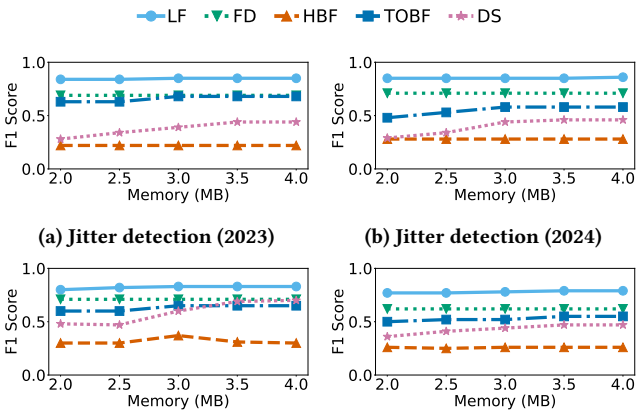


Figure 4: CDF of flow sizes.



(a) Jitter detection (2023) **(b) Jitter detection (2024)**

(c) Jitter detection (2025) **(d) Jitter detection (2026)**

Figure 5: Accuracy for arrival jitter detection.

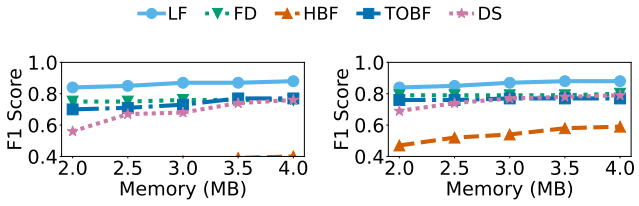
Baselines. We compare LF-Filter with Delay Sketch (DS) [41], Timeout BF (TOBF) [19], HyperBF (HBF) [26], and FD-Filter (FD) [15]. DS stores fingerprints and timestamps, replacing old timestamps when collision happens. TOBF partitions time into windows and tracks window IDs, taking the minimum ID as their estimation. HBF improves TOBF by replacing old timestamps with new timestamps to resolve collisions. FD compresses and merges aged window filters to improve accuracy.

Parameter configurations. We vary the available memory from 2 MB to 4 MB, which is moderate in view of O(10) MB hardware memory capacity [32]. By default, we fix the memory to 4 MB and invoke AwareHash. For LF-Filter, we allocate 6 hash functions to the existence filter, and allocate 10 filters to each layer. Each filter is equipped with six hash functions. Each slot uses 4-bit fingerprints and 2-bit window IDs. The predictor module uses a learning rate $\beta = 0.1$. The default IFPD threshold is $\theta = 600$ ms, and the basic window length is $T_l = \theta/(3(n_h - 1))$. Moreover, for baselines, we use the configurations reported in their original papers.

Metrics. (1) Precision denotes the ratio of the number of correctly measured high-IFPD packets to the total number of measured high-IFPD packets. (2) Recall denotes the ratio of the number of correctly measured high-IFPD packets to the total number of true high-IFPD packets. (3) F1 score is $\frac{2\times\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$. (4) Average relative error (ARE) is $\sum_{i=1}^{N_h}\frac{|\hat{D}_i - D_i|}{D_i}$, where $\hat{D}_i$ is the estimated IFPD of the i-th high-IFPD packet, D_i is the true IFPD, and N_h is the number of true high-IFPD packets. (5) Update and query speeds are measured in million instructions per second (Mips).

5.2 Experimental Results

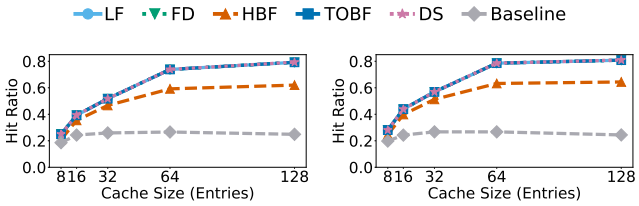
(Exp#1) Accuracy for arrival jitter detection. This application detects flows with frequently and significantly changing IFPDs. We



(a) APT detection (2023) (b) APT detection (2024)

(c) APT detection (2025) (d) APT detection (2026)

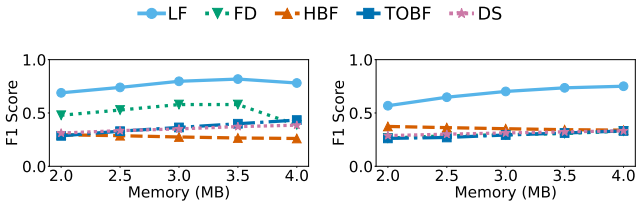
Figure 6: Accuracy for APT detection.



(a) Cache replacement (2023) (b) Cache replacement (2024)

(c) Cache replacement (2025) (d) Cache replacement (2026)

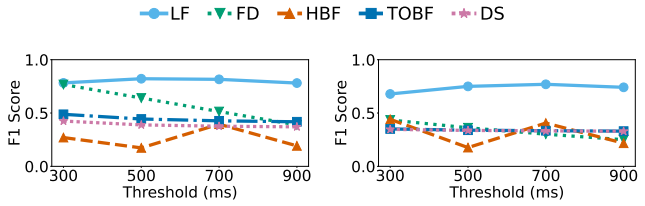
Figure 7: Accuracy for caching replacement.



(a) Accuracy vs. memory (2023) (b) Accuracy vs. memory (2024)

(c) Accuracy vs. memory (2025) (d) Accuracy vs. memory (2026)

Figure 8: Accuracy vs. different settings.



(a) Accuracy vs. thresholds (2023) (b) Accuracy vs. thresholds (2024)

(c) Accuracy vs. thresholds (2025) (d) Accuracy vs. thresholds (2026)

Figure 9: Accuracy vs. different thresholds.

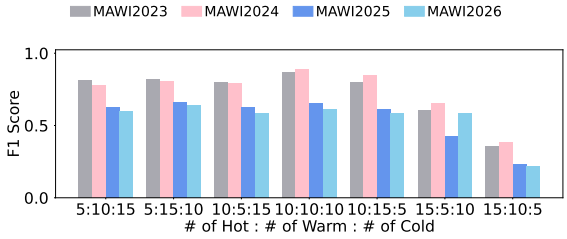
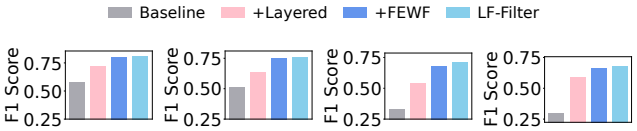


Figure 10: Sensitivity.



(a) Trace 2023. (b) Trace 2024. (c) Trace 2025. (d) Trace 2026.

Figure 11: Ablation study.

estimate IFPD and use a hash table to track the number of large IFPD changes. If the IFPD changes by more than 50% between consecutive packets, the flow introduces arrival jitters. Figure 5 indicates that LF-Filter's arrival jitter reports are accurate. It achieves the F1 score of 82.5%, whereas FD achieves 68.4%. TOBF suffers from frequent IFPD underestimation, making it miss many jitter events.

(Exp#2) Accuracy for APT detection. For APT detection, we estimate IFPDs and use a hash table to count when a small number of low-IFPD packets occur between two high-IFPD packets. If the count exceeds a threshold, we consider it as a possible attack. Here, APT detection requires distinguishing high-IFPD packets from low-IFPD bursts accurately, making LF-Filter's high detection accuracy essential. Also, its flow-aware processing particularly matches APT flows, which are often persistent and are thus heavy flows. Figure 6 shows that LF-Filter outperforms all baselines. LF-Filter achieves 85.8% F1 score, which significantly outperforms other baselines.

(Exp#3) Accuracy for caching replacement. We can enhance the hit rate of LFU with accurate IFPD detection in LF-Filter. If a key incurs low IFPD, implying active cache usage, we protect that key to remain in the cache for at least τ. We prioritize evicting unprotected

Table 4: Resource consumption of LF-Filter.

Component(s)	PHV	SRAM	TCAM	VLIW	ALU	Stage Number
Existence Filter	13.28%	7.5%	0%	3.12%	25%	4
+ Flow Filter	19.53%	12.5%	0%	3.12%	75%	4
+ Predictor	19.53%	12.5%	0%	3.12%	75%	4
+ FEWF	19.53%	12.5%	0%	3.12%	75%	8
LF-Filter	19.53%	12.5%	0%	4.69%	75%	8

Table 5: Performance of LF-Filter on switch ASICs.

Workload	20%	40%	60%	80%	100%
Throughput	1.28 Tbps	2.56 Tbps	3.84 Tbps	5.12 Tbps	6.4 Tbps
Latency	245 ns	245 ns	245 ns	245 ns	245 ns

keys that exhibit high IFPD and declining popularity. We vary the cache size from 8 to 128 entries and use the four traces to simulate memory access requests. Figure 7 shows that with LF-Filter, the hit rate improves to 16.8%-80.8%.

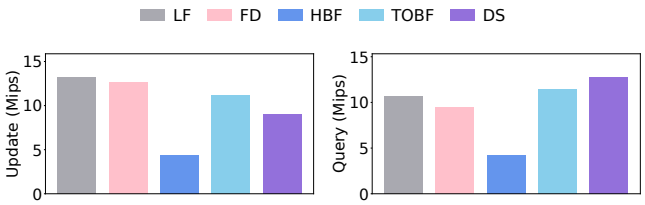
(Exp#4) Accuracy under different settings. We evaluate accuracy under different memory settings. We vary the available memory from 2 MB to 4 MB. In Figure 8 (a)-(d), LF-Filter outperforms other baselines by achieving 53.1%-82.4% F1 score. Such high accuracy originates from its efficient fingerprint-enhanced filters. In contrast, FD wastes memory to provide accurate timestamps for all flows, leading to accuracy drops. HBF's timestamp replacement results in underestimation. TOBF overestimates the IFPD of the first packet in a flow and underestimates subsequent packets.

(Exp#5) Accuracy with different thresholds. In practice, the traffic in different scenarios exhibits varying IFPD distributions, which corresponds to different thresholds to maximize the detection accuracy. Thus, to test the accuracy under different IFPD distributions, we vary the IFPD threshold θ from 300 ms to 900 ms. In Figure 9 (a)-(d), LF-Filter outperforms all baselines. For example, when $\theta = 300$ ms, LF-Filter achieves the F1 score of 79.9%, which improves 73.4% in FD and 42.3% in DS.

(Exp#6) Sensitivity. We evaluate the sensitivity of LF-Filter. Specifically, we fix the total number of filters to 30 while varying the ratio of number of filters in the hot, warm, and cold layers. In Figure 10, the accuracy of LF-Filter retains high-level with small changes in the default settings, demonstrating low sensitivity. The notion behind such robustness originates from its powerful predictor, which enables IFPD queries to rapidly locate the previous timestamp records.

(Exp#7) Ablation study. We perform ablation study towards LF-Filter when measuring the four traces. More precisely, we first disable the benefits of all data structures in LF-Filter, and then gradually activate them to observe accuracy differences. In Figure 11, each type of data structures brings remarkable accuracy improvements to IFPD measurements, demonstrating the power of careful data structure design and integration in LF-Filter.

(Exp#8) Hardware implementation. We implement LF-Filter on a Tofino switch. Its Filters are realized by registers. We configure the existence filter with 2^{20} bits, and configure hot, warm, and cold layers with the default settings. Each slot holds a 4-bit fingerprint and 2-bit window IDs, selected via bitwise masks. We abstract each function of LF-Filter as a match-action table, i.e., getting the current timestamp, computing the filter index, calculating the window ID, classifying the flow, and updating the existence filter and window



(a) Update Performance. **(b) Query Performance.**

Figure 12: Performance of LF-Filter on NVIDIA DPUs.

filter. We first measure the consumption of existence filter in LF-Filter, and then gradually add other components while capturing differences in consumption. As shown in Table 4, LF-Filter consumes 8 stages with small per-stage footprint, leaving headroom for concurrent network functions to share stages.

(Exp#9) Performance. We evaluate the performance of LF-Filter on two platforms, i.e., switch ASICs and NVIDIA DPUs.

First, we evaluate the performance of LF-Filter on switch ASICs, including throughput and per-query processing latency. We use snake testing [16], a standard way of traffic testing in industry, to stress-test the performance. The first and last ports serve as traffic ports connected to two servers, while the $(2i-1)$-th and $(2i)$-th normal ports are paired for $0 < i < \beta/2$, where β is the total number of normal ports. Next, each server injects 100 Gbps traffic to a specific traffic port while the switch forwards through all paired ports. We consider 50% of the total switch throughput as the 100% workload of a stress test because the average link utilization in production networks is typically 20%-40% [30, 38]. In Table 5, the throughput of LF-Filter remains the line rate while the per-query latency is only increased by 245 ns because switch ASIC pipelines always guarantee line-rate performance.

Second, we evaluate LF-Filter on NVIDIA DPUs. Figure 12 shows the performance of update and query in LF-Filter and baselines. In Figure 12(a), LF-Filter yields the best performance since it only uses simple operations during update. In Figure 12(b), it outperforms the state-of-the-art FD-Filter via its predictor module. TOBF and DS achieve the best due to their simple design, which in turn makes them fall short in accuracy.

6 Conclusion

IFPD is a critical metric in high-speed data streams that enables various applications such as arrival jitter detection, APT detection, and caching optimization. This paper proposes LF-Filter, a layered flow-aware data structure that provides differentiated processing for different flows to maintain accuracy while reducing resource footprints. We theoretically analyze LF-Filter's false positive rate, false negative rate, and estimation error bounds. Extensive experiments demonstrate that LF-Filter improves F1 score by up to 2.22× when compared with state-of-the-art solutions.

Acknowledgments

We thank the reviewers and the area chair for their constructive comments. This work is supported by the National Key R&D Program of China (2024YFB2906503), the National Natural Science Foundation of China (Grant No. 62602005), and the Science and Technology Program of Fujian Province (Grant No. 2025H6007).

References

[1] Subutai Ahmad and Scott Purdy. 2016. Real-time anomaly detection for streaming analytics. *arXiv preprint arXiv:1607.02480* (2016).

[2] Adel Alshamrani, Sowmya Myneni, Ankur Chowdhary, and Dijiang Huang. 2019. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Communications Surveys & Tutorials* 21, 2 (2019), 1851–1877.

[3] Theophilus Benson, Aditya Akella, and David A Maltz. 2010. Network traffic characteristics of data centers in the wild. In *ACM IMC*. 267–280.

[4] Burton H Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13, 7 (1970), 422–426.

[5] Graham Cormode and Shan Muthukrishnan. 2005. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms* 55, 1 (2005), 58–75.

[6] Hamza Dahmouni, André Girard, and Brunilde Sansò. 2012. An Analytical Model for Jitter in IP Networks. *Annals of Telecommunications* 67, 1-2 (2012), 81–90.

[7] Michael K Daly. 2009. Advanced persistent threat. *Usenix, Nov* 4, 4 (2009), 2013–2016.

[8] Mayur Datar, Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 2002. Maintaining Stream Statistics over Sliding Windows. *SIAM J. Comput.* 31, 6 (2002), 1794–1813.

[9] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. 2014. Cuckoo Filter: Practically Better Than Bloom. In *ACM CoNEXT*. 75–88.

[10] Roy Friedman, Or Goaz, and Dor Hovav. 2023. PKache: A Generic Framework for Data Plane Caching. In *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC)*. 1268–1276.

[11] Junzhi Gong, Tong Yang, Haowei Zhang, Hao Li, Steve Uhlig, Shigang Chen, Lorna Uden, and Xiaoming Li. 2018. HeavyKeeper: An Accurate Algorithm for Finding Top-k Elephant Flows. In *USENIX ATC*. 909–921.

[12] Bur Goode. 2002. Voice Over Internet Protocol (VoIP). *Proc. IEEE* 90, 9 (2002), 1495–1517.

[13] Xiangyang Gou, Long He, Yinda Zhang, Ke Wang, Xilai Liu, Tong Yang, Yi Wang, and Bin Cui. 2020. Sliding Sketches: A Framework using Time Zones for Data Stream Processing in Sliding Windows. In *ACM SIGKDD*. 1615–1625.

[14] Jiarui Guo, Yisen Hong, Yuhan Wu, Yunfei Liu, Tong Yang, and Bin Cui. 2023. SketchPolymer: Estimate Per-item Tail Quantile Using One Sketch. In *ACM SIGKDD*. 590–601.

[15] Jintao He, Jie Gui, Tian Lv, Jiaqi Zhu, and Qun Huang. 2025. FD-Filter: A Compact Data Structure for Fine-Grained Intra-Flow Packet Delay Monitoring. In *IEEE INFOCOM*. 1–10.

[16] Xin Jin, Xiaozhou Li, Haoyu Zhang, Robert Soulé, Jeongkeun Lee, Nate Foster, Changhoon Kim, and Ion Stoica. 2017. Netcache: Balancing key-value stores with fast in-network caching. In *ACM SOSP*. 121–136.

[17] Faisal Khan, Nicholas Hosein, Soheil Ghiasi, Chen-Nee Chuah, and Puneet Sharma. 2013. Streaming solutions for fine-grained network traffic measurements and analysis. *IEEE/ACM Transactions On Networking* 22, 2 (2013), 377–390.

[18] Ramana Rao Kompella, Kirill Levchenko, Alex C. Snoeren, and George Varghese. 2009. Every Microsecond Counts: Tracking Fine-Grain Latencies with a Lossy Difference Aggregator. In *ACM SIGCOMM*. 255–266.

[19] Shijin Kong, Tao He, Xiaoxin Shao, Changqing An, and Xing Li. 2006. Time-out bloom filter: A new sampling method for recording more flows. In *International Conference on Information Networking*. Springer, 590–599.

[20] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *ACM SIGMOD*. 489–504.

[21] Myungjin Lee, Nick Duffield, and Ramana Rao Kompella. 2012. High-fidelity per-flow delay measurements with reference latency interpolation. *IEEE/ACM Transactions on Networking* 21, 5 (2012), 1567–1580.

[22] Jizhou Li, Zikun Li, Yifei Xu, Shiqi Jiang, Tong Yang, Bin Cui, Yafei Dai, and Gong Zhang. 2020. WavingSketch: An Unbiased and Generic Sketch for Finding Top-k Items in Data Streams. In *ACM SIGKDD*. 1574–1584.

[23] Zhenyu Li, Mohamed Ali Kaafar, Kave Salamatian, and Gaogang Xie. 2016. Characterizing and modeling user behavior in a large-scale mobile live streaming system. *IEEE Transactions on Circuits and Systems for Video Technology* 27, 12 (2016), 2675–2686.

[24] Zhenyu Li, Gaogang Xie, Mohamed Ali Kaafar, and Kave Salamatian. 2015. User behavior characterization of a large-scale mobile live streaming system. In *ACM TheWebConf*. 307–313.

[25] Evan Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An Imitation Learning Approach for Cache Replacement. In *ICML (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 6237–6247.

[26] Zirui Liu, Chaozhe Kong, Kaicheng Yang, Tong Yang, Ruijie Miao, Qizhi Chen, Yikai Zhao, Yaofeng Tu, and Bin Cui. 2023. HyperCalm Sketch: One-Pass Mining Periodic Batches in Data Streams. In *IEEE ICDE*. 14–26.

[27] Yishay Mansour and Boaz Patt-Shamir. 2001. Jitter Control in QoS Networks. *IEEE/ACM Transactions on Networking* 9, 4 (2001), 492–502.

[28] P4.org Applications Working Group. 2020. In-band Network Telemetry (INT) Dataplane Specification. https://p4.org/p4-spec/docs/INT_v2_1.pdf. Version 2.1.

[29] Ashwin Rao, Arnaud Legout, Yeon-sup Lim, Don Towsley, Claude Barker, and Christophe Diot. 2011. Network Characteristics of Video Streaming Traffic. In *ACM CoNEXT*. 1–12.

[30] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. 2015. Inside the social network's (datacenter) network. In *ACM SIGCOMM*. 123–137.

[31] The MAWI Traces. [n. d.]. https://mawi.wide.ad.jp/mawi/.

[32] Tofino2. [n. d.]. https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch/tofino-2-series.html.

[33] Jiliang Wang, Shuo Lian, Wei Dong, Yunhao Liu, and Xiang-Yang Li. 2014. Every packet counts: Fine-grained delay and loss measurement with reordering. In *IEEE ICNP*. 95–106.

[34] Yuhan Wu, Shiqi Jiang, Siyuan Dong, Zheng Zhong, Jiale Chen, Yutong Hu, Tong Yang, Steve Uhlig, and Bin Cui. 2023. MicroscopeSketch: Accurate Sliding Estimation Using Adaptive Zooming. In *ACM SIGKDD*. 2660–2671.

[35] Tong Yang, Junzhi Gong, Haowei Zhang, Lei Zou, Lei Shi, and Xiaoming Li. 2018. HeavyGuardian: Separate and Guard Hot Items in Data Streams. In *ACM SIGKDD*. 2584–2593.

[36] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. 2018. Elastic sketch: Adaptive and fast network-wide measurements. In *ACM SIGCOMM*. 561–575.

[37] Tong Yang, Haowei Zhang, Jinyang Li, Junzhi Gong, Steve Uhlig, Shigang Chen, and Xiaoming Li. 2019. HeavyKeeper: an accurate algorithm for finding Top-k elephant flows. *IEEE/ACM Transactions on Networking* 27, 5 (2019), 1845–1858.

[38] Nofel Yaseen, John Sonchack, and Vincent Liu. 2020. tpprof: A Network Traffic Pattern Profiler. In *USENIX NSDI*. 1015–1030.

[39] Yikai Zhao, Wenrui Liu, Fenghao Dong, Tong Yang, Yuanpeng Li, Kaicheng Yang, Zirui Liu, Zhengyi Jia, and Yongqiang Yang. 2023. P4LRU: Towards An LRU Cache Entirely in Programmable Data Plane. In *ACM SIGCOMM*. 967–980.

[40] Yikai Zhao, Zheng Zhong, Yuanpeng Li, Yi Zhou, Yifan Zhu, Li Chen, Yi Wang, and Tong Yang. 2021. Cluster-Reduce: Compressing Sketches for Distributed Data Streams. In *ACM SIGKDD*. 2316–2326.

[41] Jiaqi Zhu, Kai Zhang, and Qun Huang. 2021. A Sketch Algorithm to Monitor High Packet Delay in Network Traffic. In *APNet*. 43–49.

Appendix 1: Proof

Complexity. We analyze space and time complexity as follows.

Theorem 1. *The update and query time complexity is* $O(k_e + k_w)$.

Proof. Each update requires k_e hash operations for the existence filter and k_w operations for the window filter. With counter combination, querying all filters in a layer needs only k_w hash operations. With prediction-guided search, queries typically access only one layer, yielding $O(k_e + k_w)$ complexity. □

Theorem 2. *The space complexity of LF-Filter is* $O(m_e + (f + w) \cdot m_w \cdot (n_h + n_w) + m_w \cdot n_c)$.

Proof. The existence filter uses m_e bits. Hot and warm layers use $(n_h + n_w)$ fingerprint-enhanced window filters, in which each slot has $(f + w)$ bits. The cold layer uses n_c 1-bit filters. □

False positive rate. A false positive occurs when a low-IFPD packet is wrongly identified as high-IFPD.

Lemma 1 (Packet Arrival Distribution). *For Poisson arrivals with rate* λ*, if the* c*-th packet arrives at* t_c*, the probability that IFPD exceeds* v *is* $p_v = (1 - v/t_c)^c$.

Lemma 2 (Collision Probability). *For fingerprint-enhanced filters, the effective collision probability, i.e., colliding at the same slot and matching fingerprints, is* $P_{col} = \frac{1}{m_w \cdot 2^f}$.

Theorem 3 (False Positive Rate). *Let* N_i *be the number of flows in a filter. The false positive rate is:*

$$FPR_{LF} = \left(1 - \left(1 - \frac{1}{m_w \cdot 2^f}\right)^{N_i}\right)^{k_w} \approx \left(\frac{N_i}{m_w \cdot 2^f}\right)^{k_w} \tag{9}$$

Compared to FD-Filter's $FPR_{FD} \approx (N_i/m_w)^{k_w}$, the reduction is:

$$\frac{FPR_{FD}}{FPR_{LF}} = 2^{f \cdot k_w} \tag{10}$$

For $f = 4$ bits and $k_w = 3$, this yields a $2^{12} = 4096\times$ reduction.

Proof. A false positive requires all k_w mapped slots to have collisions. For each slot, the probability that at least one of N_i flows causes a collision is $1 - (1 - \frac{1}{m_w \cdot 2^f})^{N_i}$. □

False negative rate. A false negative occurs when a high-IFPD packet is wrongly identified as low-IFPD.

Lemma 3 (Layer Hit Probability). *For high-IFPD packets with exponential delay distribution at rate μ, the probabilities of predecessor in each layer are*

$$p_h = 1 - e^{-3\mu n_h T_l}, \quad p_w = e^{-3\mu n_h T_l}(1 - e^{-6\mu n_w T_l}), \quad p_c = 1 - p_h - p_w \tag{11}$$

Theorem 4 (Slot Retention Probability). *Let $\bar{c} = N_i/m_w$ be the average collisions per slot. The probability that a slot retains flow x's information is:*

$$P_{retain} = \left(1 - \frac{1}{m_w}\right)^{N_i - 1} + \frac{1}{m_w} \cdot \left(\frac{1}{2^f} + \left(1 - \frac{1}{2^f}\right) \cdot \frac{1}{\bar{c} + 1}\right) \tag{12}$$

Proof. With probability $(1 - 1/m_w)^{N_i - 1}$, no other flow maps to the slot and x is retained. Otherwise, among colliding flows: (1) with probability $1/2^f$, fingerprints match and window IDs merge via OR without loss; (2) with probability $1 - 1/2^f$, fingerprints differ and probabilistic replacement occurs with probability $\approx 1/(\bar{c} + 1)$ of retaining x. □

Theorem 5 (False Negative Rate). *The false negative rate of LF-Filter is bounded by:*

$$FNR_{LF} = \sum_{l \in \{h,w,c\}} p_l \cdot (1 - P_{retain,l}^{k_w}) \tag{13}$$

where p_l is from Lemma 3 and $P_{retain,l}$ is from Theorem 4.

Error. We analyze the estimation error of LF-Filter.

Theorem 6 (Expected Estimation Error). *The expected IFPD estimation error is:*

$$\mathbb{E}[\epsilon] = p_h \cdot \frac{T_l}{4} + p_w \cdot \frac{T_l}{2} + p_c \cdot T_l \tag{14}$$

where $T_l/4$ is the expected error within a window of length T_l (from midpoint estimation). For $p_h = 0.7$, $p_w = 0.2$, $p_c = 0.1$, and $n_h = 6$, the ARE bound is $\approx 2.5\%$.

Prediction-guided query. The prediction module uses exponential moving average (EMA) to learn flow patterns. Let $\hat{l}_n$ be the predicted layer after n packets and l^* be the true layer. The EMA update $\hat{l}_n = (1 - \beta)\hat{l}_{n-1} + \beta l^*$ converges to l^* within $O(\frac{1}{\beta} \ln \frac{2}{\epsilon})$ packets for ϵ-accuracy. With prediction accuracy α, the expected number of searched layers is $\mathbb{E}[L] = \alpha \cdot 1 + (1 - \alpha) \cdot 2$, e.g., when $\alpha = 0.85$, $\mathbb{E}[L] = 1.15$ compared to sequential search's $\mathbb{E}[L_{seq}] = p_h + 2p_w + 3p_c \approx 1.4$, which is a 1.22× speedup.

Parameter guidelines. Given the target FPR ϵ and flow count N_i, the filter size should satisfy $m_w \geq N_i \cdot 2^f / \epsilon^{1/k_w}$. Following bloom filter theories, $k_w = 0.693 \cdot m_w \cdot 2^f / N_i$ minimizes FPR while $k_w = 3$ balances accuracy and speed.

Appendix 2: Additional Evaluation

Generalization. We additionally evaluate LF-Filter on two streaming traces from other domains, i.e., a data center trace and an IoT sensor trace. As shown in Table 6, LF-Filter achieves F1 scores of 0.6550 on the DC trace and 0.7535 on the IoT trace, outperforming the state-of-the-art FD-Filter under the same memory budget. These results indicate that LF-Filter's advantage extends to diverse streaming workloads beyond network traffic.

Table 6: Generalization beyond backbone network traffic.

Trace	LF-Filter F1	FD-Filter F1	Improvement
DC	0.6550	0.4677	18.73%
IoT	0.7535	0.0323	72.12%

Robustness. We further test LF-Filter on synthetic Zipf workloads with skewness from 0.6 to 1.4 to examine its behavior when the heavy-tailed assumption weakens. As shown in Table 7, LF-Filter consistently outperforms FD-Filter across the full range, confirming that its gains do not rely on extreme skew.

Table 7: Robustness under varying Zipf skewness.

Zipf skewness	LF-Filter F1	FD-Filter F1	Improvement
0.6	0.9443	0.9264	1.79%
0.8	0.9239	0.8696	5.43%
1.0	0.9019	0.7418	16.01%
1.2	0.8770	0.5617	31.53%
1.4	0.8335	0.3888	44.47%